pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Loss Functions and the Learning Rate

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 2: training, on a number

Day 6: Loss functions and the learning rate

Day 7: Momentum, Adam, schedules

Day 8: Initialization, batch normalization

Day 9: Overfitting, dropout, early stopping

Day 10: Case study: class weights

The benchmark rules

821 bonds of invented issuers; spreads from the course's generator. Course 4's 206 test bonds stay closed: every number comes from the 615 training bonds. A fitted value is the network's spread for these bonds, never a fair value.

Today's question: on bonds a straight line already describes, what does a network add?

A linear output is a linear regression

```
keras.Sequential([keras.Input(shape=(29,)), keras.layers.Dense(1)])   # no activation
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1), loss="mse")
```

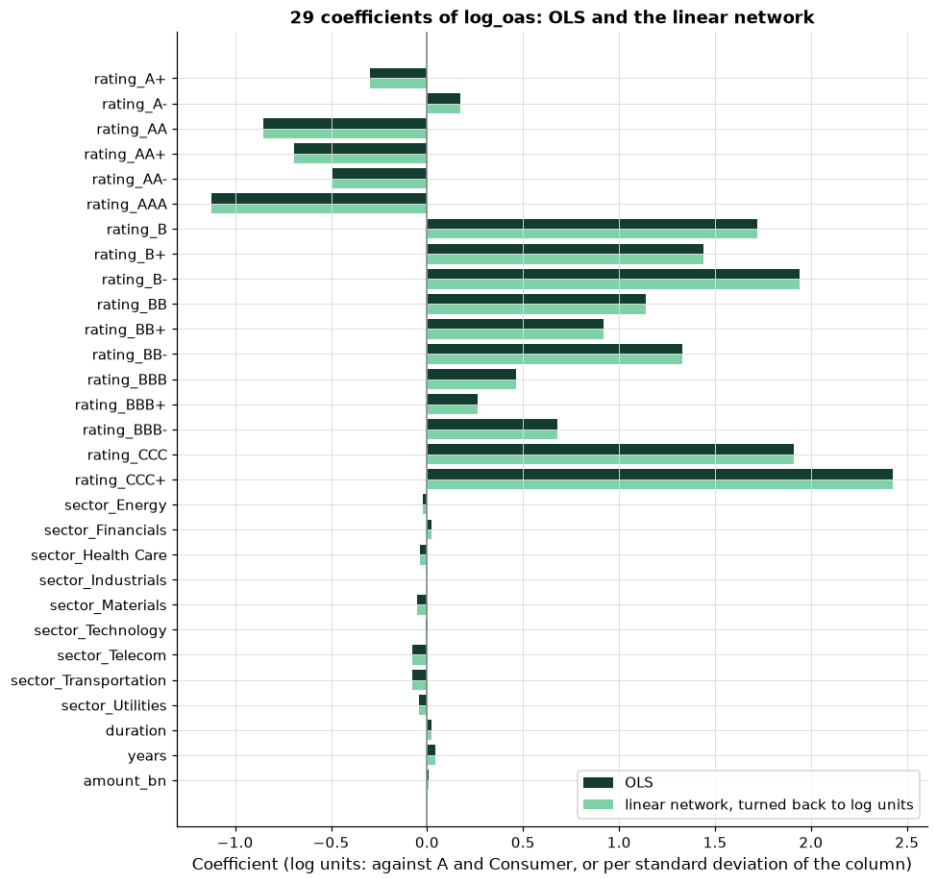
Coefficient	OLS	Network
intercept	4.3488	4.3488
rating_BBB	0.4624	0.4624
rating_BB	1.1381	1.1381
duration	0.0244	0.0244
years	0.0414	0.0414
amount_bn	0.0067	0.0067

30 parameters, fitted on all 615 bonds

Every coefficient within 1e-4 of OLS's; every fitted spread within 0.01 bp. Mean squared error 0.061982 against OLS's 0.061982 (log units).

Dense(1), no activation, MSE loss: the equation and the loss of OLS.

Every coefficient, both fits



Two bars, one length

Dark green OLS, light green the network turned back to log units. Every pair matches: the network and OLS are one line.

The ratings carry the fit

Long bars for the rating dummies against A; short ones for the sectors and the three numbers.

Three losses, and the largest misses

```
MSE    average of e^2                                OLS's loss
MAE    average of |e|
Huber  0.5 e^2 if |e| <= delta, else delta (|e| - 0.5 delta)
```

On the linear network's fitted values (log units): MSE 0.0620, MAE 0.1887, Huber 0.0306.

18.3%

of the squared error

17.3%

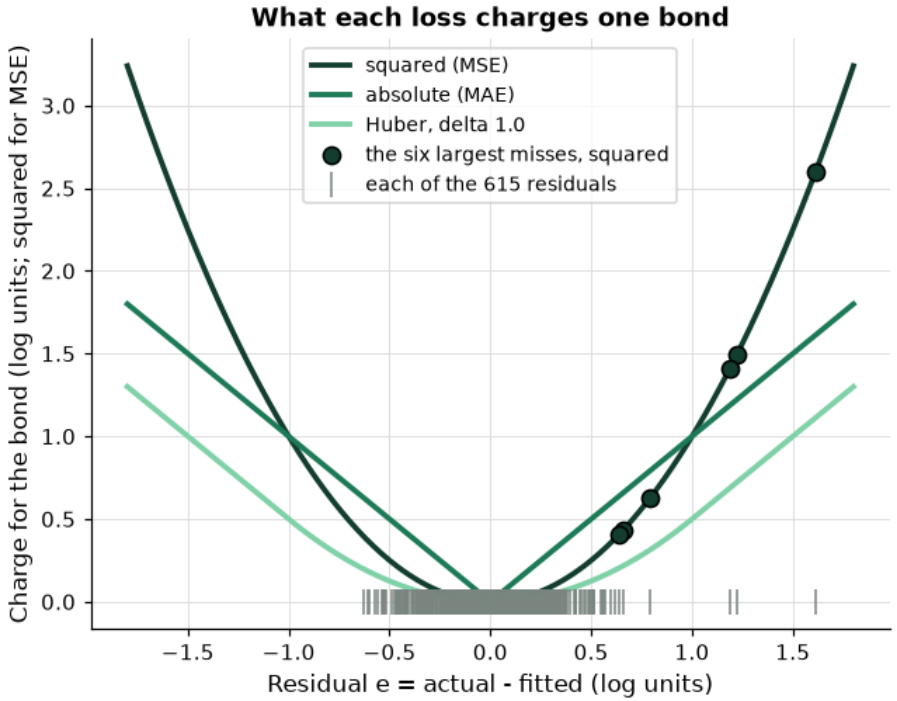
of the Huber loss

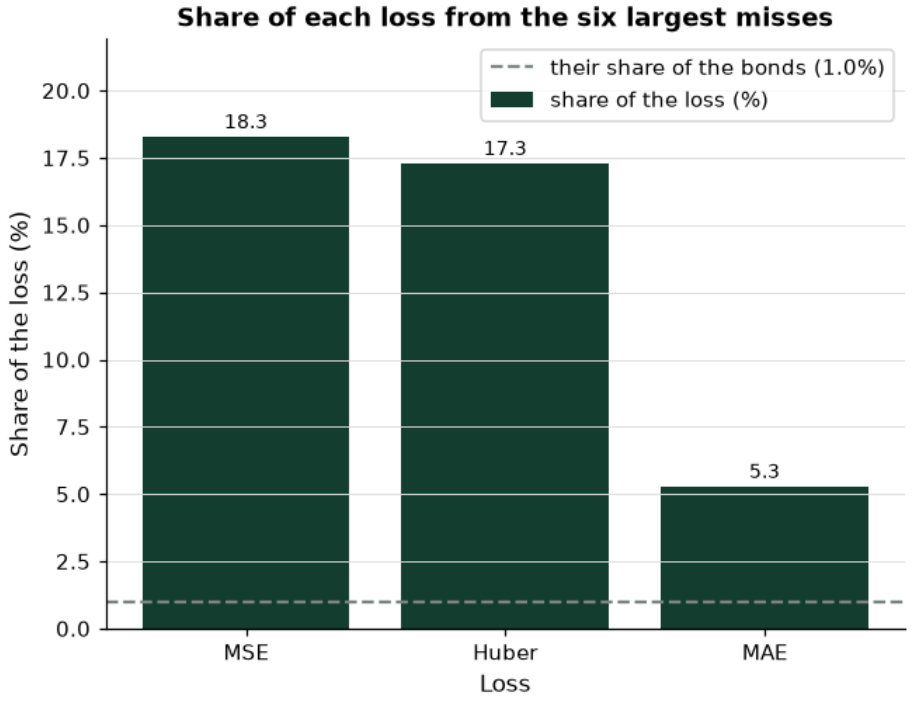
5.3%

of the absolute error

Six bonds of 615. MSE leans hardest on the largest misses; MAE least; Huber between.

What each loss charges one bond





Past a residual of about 1 the squared curve climbs away; six bonds are 1 percent of the rows.

Standardizing the target, and turning it back

24.8203

first-epoch loss, raw log_oas

1.2214

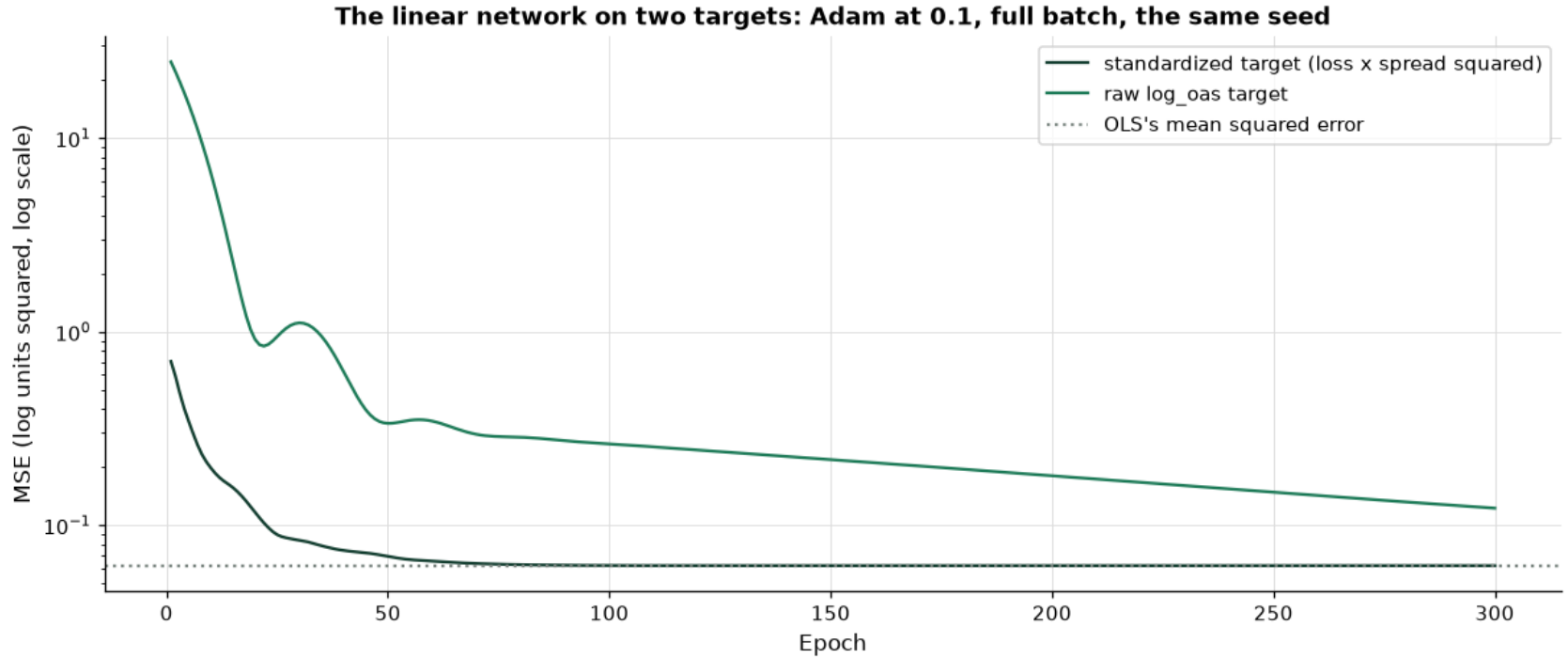
first-epoch loss, standard units

Raw target after the same 300 epochs: largest coefficient gap 1.2767; one fitted spread 236.52 bp from OLS's.

```
t = (log_oas - centre) / spread        # centre 4.8776, spread 0.7588 here
fitted_log = output * spread + centre   # turn back before anything is reported
```

In cross-validation, centre and spread come from each fold's fit rows only.

Two targets, one loss curve



Same units, same seed, same 300 epochs: the standardized run reaches OLS's line; the raw run does not.

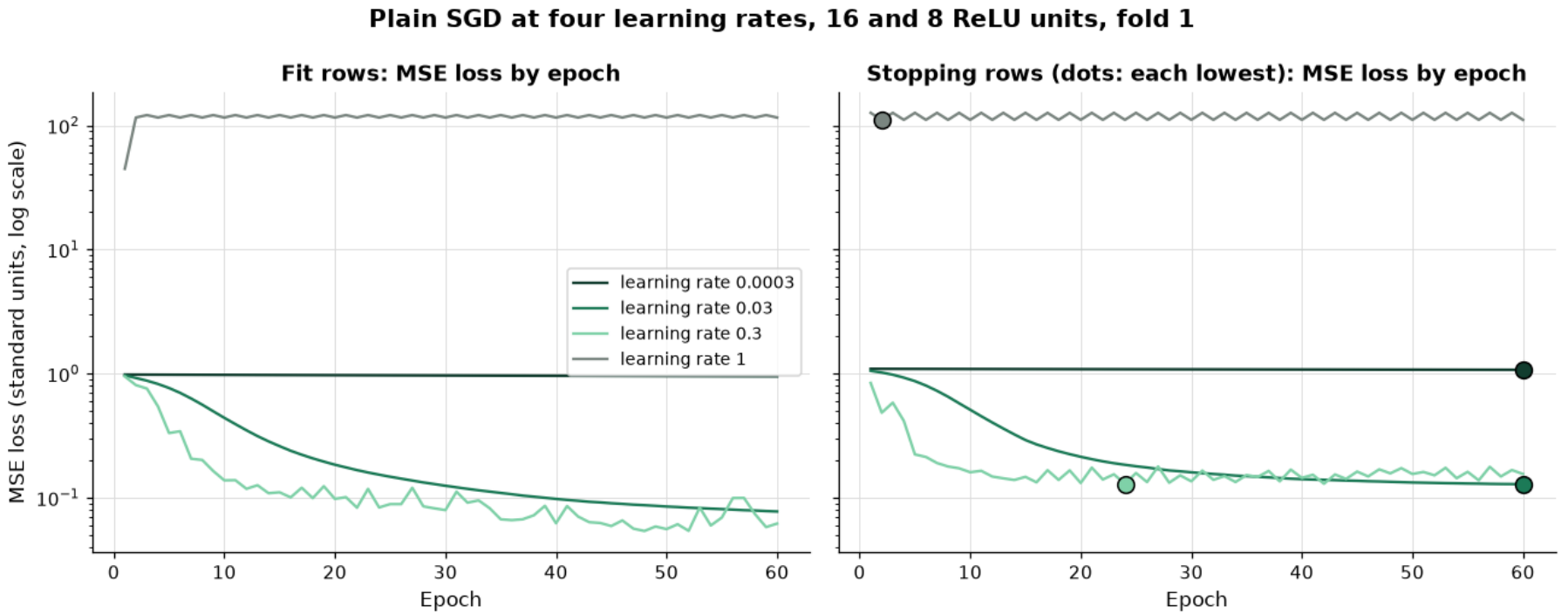
The learning rate, on one fold

Learning rate	Fit loss, ep. 1	Fit loss, ep. 60	Stopping, ep. 60	Lowest stop (epoch)	Reading
0.0003	0.9849	0.9527	1.0754	1.0754 (60)	too small
0.03	0.9707	0.0777	0.1287	0.1287 (60)	about right
0.3	0.9504	0.0620	0.1557	0.1296 (24)	fast, then overfits
1	44.7752	115.8892	111.0100	111.0100 (2)	too large

16 and 8 ReLU units, plain SGD, batch 32, MSE in standard units; fold 1's fit and stopping rows.

One setting changed between the runs: the learning rate. A loss of 1 is what the mean alone gives.

The loss curves



Watch both curves: the fit rows say whether it learns; the stopping rows say when to stop.

cv_regressor: five folds, written once

1. A random fifth of the fold's training rows to stop on
2. assert_disjoint: fit and stopping rows never touch the held-out fold
3. fit_arrays: the encoder sees the fit rows only
4. The target standardized with the fit rows' mean and standard deviation
5. Seed, build, fit with early stopping on the stopping rows
6. **Held-out output turned back: output * spread + centre**

Returns one row per fold (mae, r2, best_epoch) and the out-of-fold fitted values. 28 tests pass.

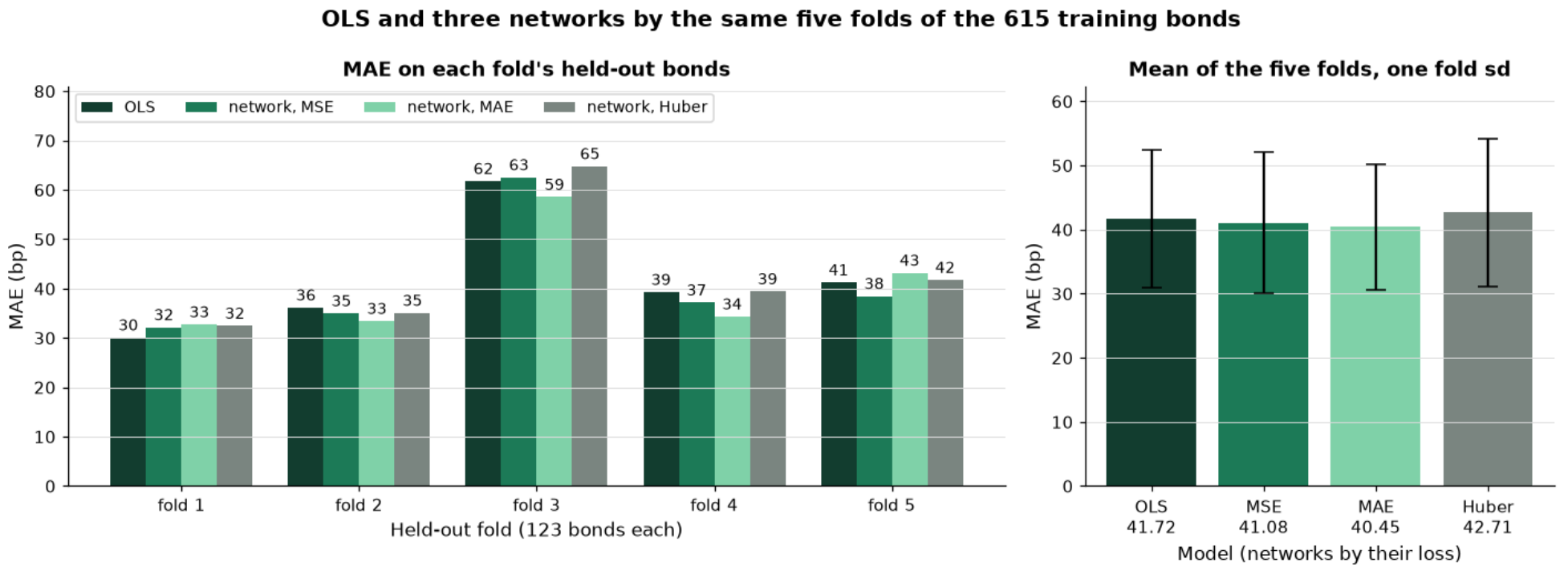
Three networks beside OLS

Model	MAE (bp)	Fold sd (bp)	R-squared	Folds below OLS
OLS	41.72	10.72	0.8794	
Network, MSE loss	41.08	10.95	0.8659	3 of 5
Network, MAE loss	40.45	9.81	0.8720	3 of 5
Network, Huber loss	42.71	11.51	0.8647	1 of 5

16 and 8 ReLU units, Adam at 0.01, early stopping; the same five folds of 123 bonds for every model. OLS: 41.72 bp.

One setting changed between the networks: the loss. The fold spread is several times every gap.

Fold by fold, four models



The folds differ far more than the models: every model misses most on fold 3 and least on fold 1.

What the numbers support

Fold	OLS (bp)	Network, MAE (bp)	Difference
fold 1	30.09	32.75	2.66
fold 2	36.17	33.43	-2.74
fold 3	61.78	58.56	-3.22
fold 4	39.30	34.34	-4.96
fold 5	41.24	43.19	1.95

Below OLS in 3 of 5; mean difference -1.26 bp.

A little flattering

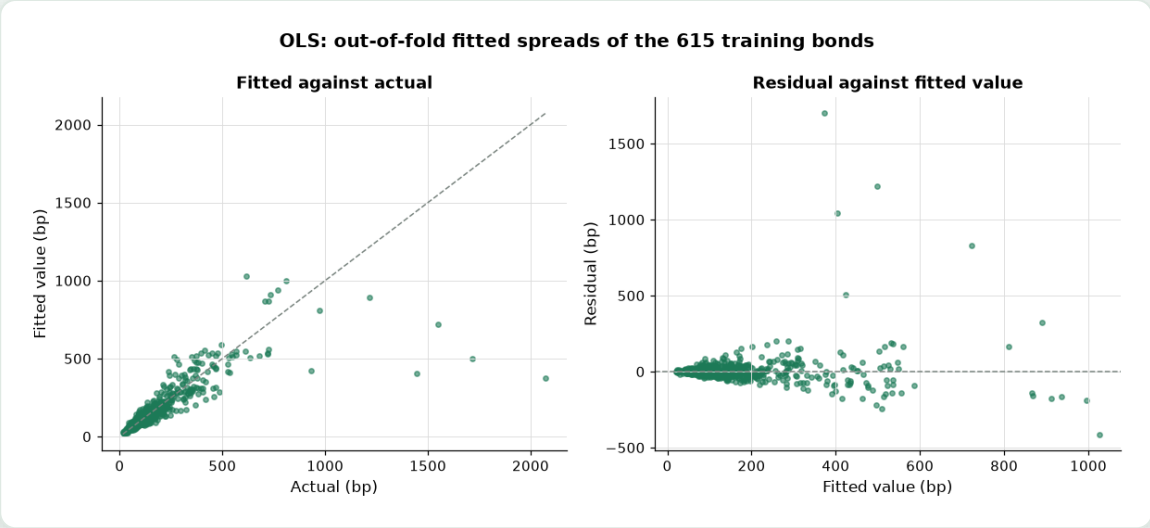
The same five folds compared three losses and report the winner.

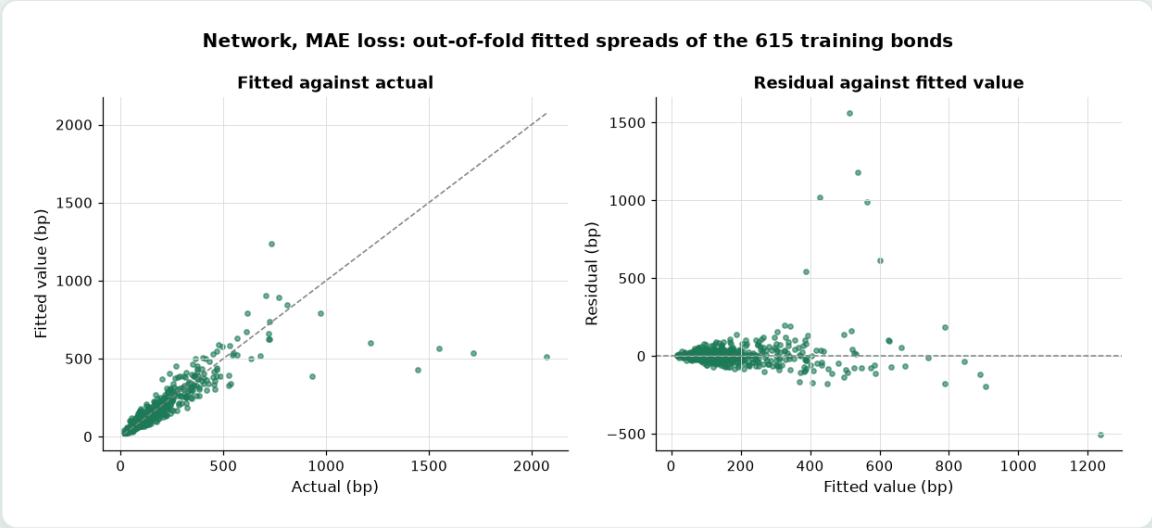
Why so close

The generator multiplies its parts, so in logs they add: a linear shape.

About as well as OLS, not clearly better.

Residuals in bp





OLS on the left, the MAE network on the right; every fitted spread from a model that did not see the bond.

The two look alike. The residuals fan out as the fitted spread widens: a miss in logs is a share of the spread.

Units: the MAE in bp

0.2487

MAE, standard units

0.1887

MAE, log units

39.35

MAE in bp, after exp

One set of fitted values (section 6.1, the bonds it was fitted on). Only the last is in bp.

Allowed names

fitted spread

the network's spread for these bonds

residual

Never called

fair value or fair spread

cheap, rich, mispriced

a signal or a trade

Excel side by side

Excel side by side (tested in Excel)

`=SUMXMY2(A2:A616,B2:B616)/COUNT(A2:A616)`	MSE 0.061982
`=AVERAGE(ABS(A2:A616-B2:B616))`	MAE 0.188728
`=IF(ABS(C2)<=$H$1,0.5*C2^2,$H$1*(ABS(C2)-0.5*$H$1))`	Huber row, filled down (delta in H1)
`=AVERAGE(D2:D616)`	Huber 0.030620
`=AVERAGE(ABS(EXP(A2:A616)-EXP(E2:E616)))`	MAE 41.72 bp (OLS, out of fold)
`=AVERAGE(A2:A616) and =STDEV.P(A2:A616)`	4.8776 and 0.7588
`=(A2-$K$1)/$K$2 and =N2*$K$2+$K$1`	standard units, and back

No network training in Excel. Every loss and every change of units is a formula, equal to Python's within 1e-12.

Save the day: the regression log

1. Morning

The four module files as day 5 left them

2. Code

regression_losses and cv_regressor

3. Results

results.csv, five regression columns

4. log -p

git log -p -1 -- netkit.py

```
model,features,rows,mae_bp,r2
ols,rating_sector_duration_years_amount,cv,41.7166,0.8794
network_16_8_mse,rating_sector_duration_years_amount,cv,41.0759,0.8659
network_16_8_mae,rating_sector_duration_years_amount,cv,40.4533,0.8720
network_16_8_huber,rating_sector_duration_years_amount,cv,42.7051,0.8647
```

One file holds one kind of row: a regression log has its own columns.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: output left in standard units: it reports 0.39 bp; turned back, the network misses by 33.28 bp.

Code that runs is not the same as code that is right.

What you can do now

- Show a linear output with MSE is a linear regression
- Compute MSE, MAE, and Huber by hand
- Standardize a target and turn it back
- Read the learning rate off the loss curves
- Score a network by five folds beside OLS, in bp

Week 2, Day 2 Momentum, Adam, and learning-rate schedules